

알고리즘



학습 목표

기본 알고리즘을 설계하고
AI를 활용해 코드를 작성한다.

- ❑ K-Programming
- ❑ 데이터 검색과 정렬
- ❑ 기본 알고리즘

Review

Computational Thinking

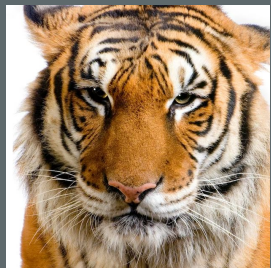
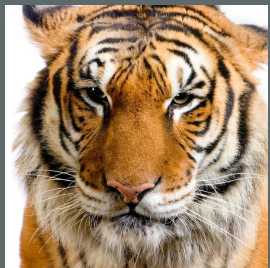
컴퓨터의 관점에서 이해하고 생각

- 문제 분해
- 패턴 인식
- 자료 표현
- 일반화/추상화
- 알고리즘

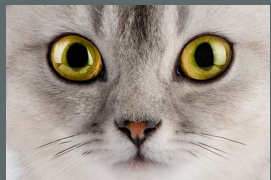
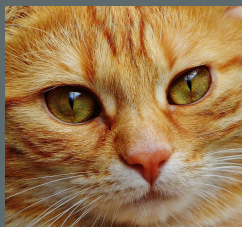




?



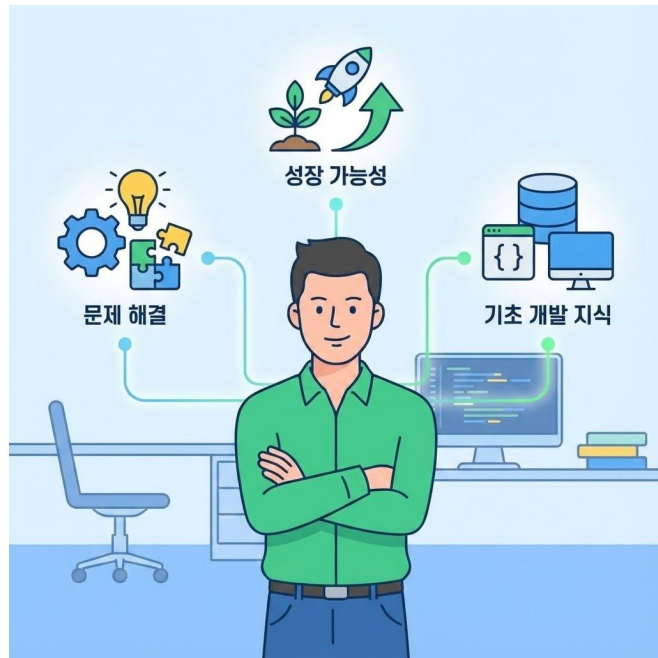
?



?

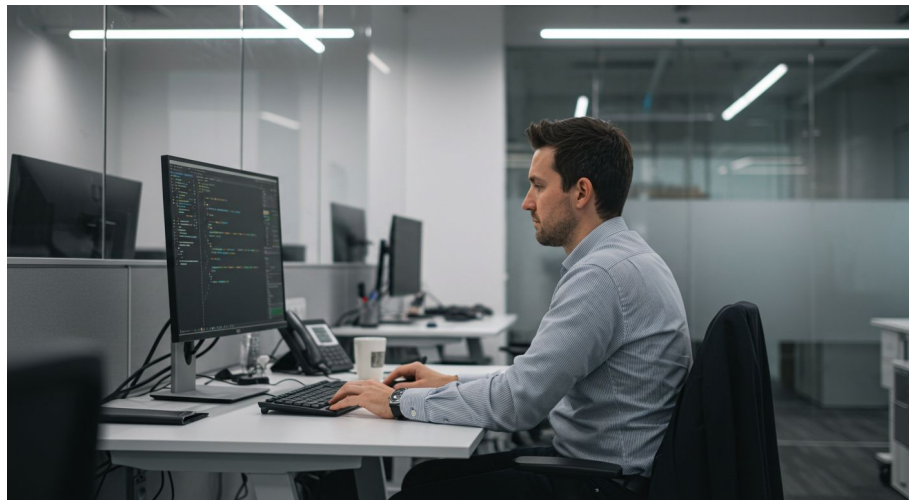
Software Engineer

- 문제 해결 능력
- 기초 개발 지식
- 지속적인 학습
- 성장 가능성
- [협업 능력]
- + 프롬프트 활용



Software Engineer

- 작게 시작
- 목표 설정
- 데드라인 설정
- 문제 해결이 어려운 경우 도움 요청



AI!



Daily Scrum

Daily Scrum

- 1 min. Check-in
 - 어제 어땠고,
 - 지금 어떨고,
 - 오늘 기대하는 것,
 - 개인 과제 Notion
 - 어려웠던 것
 - 배운 것
 - 과제 피드백
-

Break time :)

K Programming

K Programming

- 단계별로 설명하기
 - ◆ 어린아이가 이해가능한 수준
- 직접 실행해보기
 - ◆ 글에 없는 내용 행동하지 않기
- 간결하게 설명하기
 - ◆ 없어도 되는 문장 정리하기

K Programming

[예시] 합격/불합격 판독

- 시험 점수를 기억합니다.
- 시험 점수와 합격 기준 점수를 비교합니다.
- 시험 점수가 더 크거나 같으면 '합격'이라고 말하고, 내 점수가 더 작으면 '불합격'이라고 말합니다.

K Programming

주어진 서로 다른 5개의 수에서 큰 수 찾기

→ 12, 51, 21, 42, 33

→ 524523, 121412, 12412, 424, 1412412

→ 1, 4, 3, 2, 5

→

K Programming

주어진 무작위 5개의 숫자를 오름차순으로 정렬하기

→ 2, 5, 4, 1, 3

→ 5, 4, 3, 2, 1

→ 1, 2, 3, 4, 5

→

K Programming

로마숫자에서 큰 수 찾기

→ XXVIII, XXVI, XXIX, XXX, XXXI

→ XIX, XXI, XIV

K Programming

최소 동전 개수

- 손님에게 돈을 거슬러 주어야 합니다. 500원, 100원, 50원, 10원짜리 동전이 아주 많이 있습니다. 동전의 개수를 최소한으로 사용하여 거스름돈을 주는 절차를 설계하세요.
- 4720원 -> 13개
- 500원 -> 1개
- 440원 -> 8개

Lunchtime :)

간단 꿀팁

디버깅

→ 버그

- ◆ 버그 != 컴파일 에러
- ◆ 버그 == 오류 발생
- ◆ 버그 == 예상과 다른 결과
- ◆ 설계 실수 || 런타임 코드 오류 발생

디버깅

→ Debugging

◆ 버그가 발생하면 버그를 찾아 해결하는 과정

→ 프로그래밍 진행 과정

◆ Design(설계) >> Coding(구현) >> Test(테스트) >> Debug(디버깅)

디버깅

에러가 발생한 경우,

1. 당황하지 않기
2. 에러메세지 읽기
 - a. 에러 메세지: 주로 에러 설명과 함께 에러 발생 위치 정보 제공
3. 발생 코드 확인하기

디버깅

디버깅 진행,

1. 당황하지 않기
2. 원하는 결과와 다른 경우, **단계별로 확인** 하기
 - a. 정상적으로 동작하는 코드에서부터 한 줄 한 줄 천천히 확인
3. **출력 디버깅, 디버거 활용** 하기
 - a. 코드의 어느 위치에서 문제가 있는지 찾는 과정
 - b. 변수에 값을 바꿀 때마다 **값을 출력**
 - c. **예상과 다른 값이 나오는 위치를 찾아 원인 파악**

디버깅

고민해도 버그가 보이지 않는 경우,

→ 터널비전 탈출하기

◆ 터널 속으로 들어간 것처럼 시야가 극도로 좁아지는 것

디버깅

고민해도 버그가 보이지 않는 경우,

→ 질문하기

- ◆ 문제에 대해 고민해본 질문은 많이할수록 좋아요
- ◆ 질문하다보면 새로운 관점으로 문제를 보게 되어 답을 찾을 수도 있어요
- ◆ 훈수 둘 때 판이 더 잘 보이는 것 처럼, 다른 사람에게는 금방 보일 수 있어요

→ 생각 환기하기

- ◆ 스트레칭도하고 물 한 잔하면서 잠시 버그 생각에서 벗어나보세요

최초의 버그

9/9

0800 Antam started
1000 stopped - antam ✓

1300 (033) MP-MC { 1.2700 9.037 847 025
2.130476415 9.037 846 795 correct
033 PRO 2 2.130476415
correct 2.130676415

Relays 6-2 in 033 failed special speed test
in relay 11.00 test.

1700 Started Cosine Tape (Sine check)
1525 Started Multi Adder Test.

1545 Relay #70 Panel F
(moth) in relay.

First actual case of bug being found.

1630 Antam started.
1700 closed down.

Relay 2145
Relay 33



정렬

정렬

- 데이터 정렬의 목적 → 탐색
- 정렬된 데이터는 이진탐색(Binary Search) 가능

정렬

40억 개의 데이터에서 사용자 id 찾기

→ 정렬이 안되어 있는 경우

◆ 최악: 40억 번 확인해야함.

→ 정렬이 되어있는 경우

◆ 최악: 32회

정렬 알고리즘

- 선택 정렬(Selection Sort)
- 삽입 정렬(Insertion Sort)
- 버블 정렬(Bubble Sort)
- 병합 정렬(Merge Sort)
- 퀵 정렬(Quick Sort)

K Programming

→ 정렬 알고리즘 작성요청

'정렬 알고리즘 1개를 추가 설명없이 Python과 Java 코드만 제공해줘'

K Programming

- 작성한 글을 활용해 AI 에게 코드로 작성 요청
 - ◆ '작성한 단계의 글을 개선없이 그대로 주석과 함께 Python과 Java 코드로 제공해줘'

Algorithm

알고리즘

- 문제를 해결하는 **절차나 방법**, 해결방법을 정의한 단계적 절차
- input에 따라 **원하는 output**이 나오는 코드 작성
- 표현방식 > 자연어, 의사코드, 순서도, 프로그래밍언어

알고리즘

알고리즘 조건

- 입력: 외부 제공 데이터의 존재
- 출력: 적어도 하나의 결과 생성
- 명확성: 알고리즘 각 단계는 명확해야 함
- 유한성: 유한한 횟수를 진행하여 문제 해결 후 종료되어야 함

알고리즘

문제 정의 → 모델 고안 → 명세 작성 → **알고리즘 설계** → 검증 → 성능 분석 → 구현 → 테스트

알고리즘

→ 알고리즘 성능 분석

- ◆ 알고리즘 실행에 필요한 시간, 메모리 공간 같은 자원의 수를 결정하는 과정
- ◆ 복잡도(Complexity)를 계산해 분석
- ◆ 시간 복잡도, 빅오 표기법을 이용해 표현

알고리즘

→ 시간복잡도

- ◆ 알고리즘을 수행하기 위해 필요한 연산 횟수를 수치화
- ◆ 입력에 따른 알고리즘의 실행 시간의 상관관계 표현
- ◆ 빅오 표기법(Big-O), 세타 표기법(Big- Θ), 오메가 표기법(Big- Ω)

알고리즘

→ 빅오 표기법

- ◆ 알고리즘의 최악의 실행시간을 표기
- ◆ 대표적으로 사용되는 방법
- ◆ 효율적인 성능 예측

$$O(1) \rightarrow O(\log n) \rightarrow O(n) \rightarrow O(n \log n) \rightarrow O(n^2) \rightarrow O(2^n) \rightarrow O(n!)$$

알고리즘

- 무차별 대입(Brute Force)
 - ◆ 조합 가능한 모든 경우의 수를 하나씩 대입해보는 방식
 - ◆ 알고리즘 설계의 가장 기본적인 접근 방식
- 탐욕법(Greedy Algorithm)
- 분할정복(Divide and Conquer)
- 동적 계획법(DP; Dynamic Programming)

알고리즘

좋은 알고리즘

- 속도가 빠르다.
- 이해하기 쉽다.
- 재사용이 쉽다.
- 메모리 효율적이다.

Practice

[빠진 숫자 찾기]

→ 1부터 n 까지의 서로 다른 숫자중 $n-1$ 개가 순서대로 주어졌을 때 없는 숫자 찾기

◆ input: [1,2,3,5] output: 4

◆ input: [1,2,3,4,5,6,7,8,9,10,12,13,14,15] output: 11

Practice

[연속된 짝수 합]

→ 1부터 n까지의 숫자 중 짝수의 합 구하기

- ◆ input: n == 5 ; output: 6
- ◆ input n == 10 ; output: 30

Practice

[두번째로 큰 수]

→ 주어진 n개의 수에서 두번째로 큰 수 찾기

◆ input: [1,2,3,4,5] ; output: 4

◆ input: [5,4,1,13,52,1251,522,323] ; output: 522

Assignment

검색 및 정리

- Pseudo Code 의사코드
- Flowchart 순서도
- Space Complexity 공간 복잡도
- Divide and conquer 분할 정복
- DP; Dynamic Programming 동적 계획법

성찰일지 작성